FixLink — Comprehensive Project Documentation

Full Name: MIT-WPU Vyas Smart-Room Maintenance Tracker
Codename: FixLink-V1
Type: Full-stack server-rendered web application
Institution: MIT-WPU (Maharashtra Institute of Technology, World Peace University)

1. Project Overview

FixLink is an **end-to-end smart room maintenance tracking platform** built for the **Vyas Building** at MIT-WPU. It provides a complete lifecycle for facility maintenance — from a student reporting a broken projector via an interactive visual floor map, through admin triage and assignment to a qualified technician, all the way to real-time job tracking, completion verification, and analytics-driven insights.

Core Value Proposition

Capability	Description
Interactive Floor Map	Students/faculty click rooms on a visual CSS grid map instead of filling dropdowns
QR Code Access	Each room has a scannable QR code that auto-fills the report form
Multi-role System	Students, Faculty, Admins, Professionals (technicians), and a Super Admin (developer)
Real-time Notifications	Pusher-powered WebSocket events + Web Push (native browser) notifications
Professional Workflow	Full task lifecycle: Assignment → Start → Complete (with photo proof) or Cancel
Analytics Dashboard	Predictive asset health scoring, technician efficiency (MTTR), trend analysis

Capability	Description
Faculty Room Scheduling	Timetable management + ad-hoc room claiming for vacant classrooms
PDF / CSV Reports	Exportable maintenance performance reports with charts (Matplotlib)
Real-time Chat	Bidirectional messaging between Admins and Professionals

---

2. Tech Stack

Backend

Technology	Purpose
Python 3.11	Core language
Flask 3.0+	Web framework (Application Factory pattern)
Flask-SQLAlchemy	ORM for database interaction
Flask-Migrate (Alembic)	Database schema migrations
Flask-WTF / CSRFProtect	CSRF protection for all forms
Flask-Limiter	Rate limiting (1000/day, 100/hour per IP)
Flask-Caching	In-memory caching (`SimpleCache` , 1-hour TTL)
Werkzeug	Password hashing (`generate_password_hash` / `check_password_hash`)
itsdangerous	Signed token generation for email verification & password reset
Gunicorn + Eventlet	Production WSGI server (Docker deployment)

Database

Technology	Purpose
PostgreSQL (via Supabase)	Primary production database
SQLite (in-memory)	Test-only fallback
psycopg2-binary	PostgreSQL adapter for Python

Frontend

Technology	Purpose
Jinja2 Templates	Server-side rendering
Bootstrap 5.3	CSS framework + responsive grid
Bootstrap Icons	Icon library
GSAP 3.12	Smooth animations (theme transitions, page elements)
Pusher.js (client)	Real-time WebSocket subscription
Custom CSS (`style.css` — 45KB)	MIT-WPU branded styles, dark/light mode, interactive map
Vanilla JavaScript	Frontend logic, AJAX calls, notification system

External APIs & Services

Service	Purpose	Integration Point
Pusher	Real-time bidirectional communication (WebSockets)	`app/realtime.py` — server-side trigger; client-side via Pusher.js
EmailJS	Transactional email (ticket updates, verification, password reset)	`app/utils.py` — REST API calls to `api.emailjs.com`
Supabase	Managed PostgreSQL database hosting	`DATABASE_URL` in `.env`
Web Push API (VAPID)	Native browser push notifications	`pywebpush` library in `app/utils.py`

DevOps & Deployment

Technology	Purpose
Docker	Containerized deployment (Python 3.11-slim)
Docker Compose	Single-command orchestration
Vercel	Serverless deployment option (`api/index.py` entry point)
Git / GitHub	Version control

Utility Libraries

Library	Purpose
Pillow	Image processing for uploads
qrcode	QR code generation per room
fpdf2	PDF report generation
pandas	Data manipulation for exports
matplotlib	Pie chart generation embedded in PDF reports
requests	HTTP client for EmailJS API calls
python-dotenv	`.env` file loading

---

3. Application Architecture

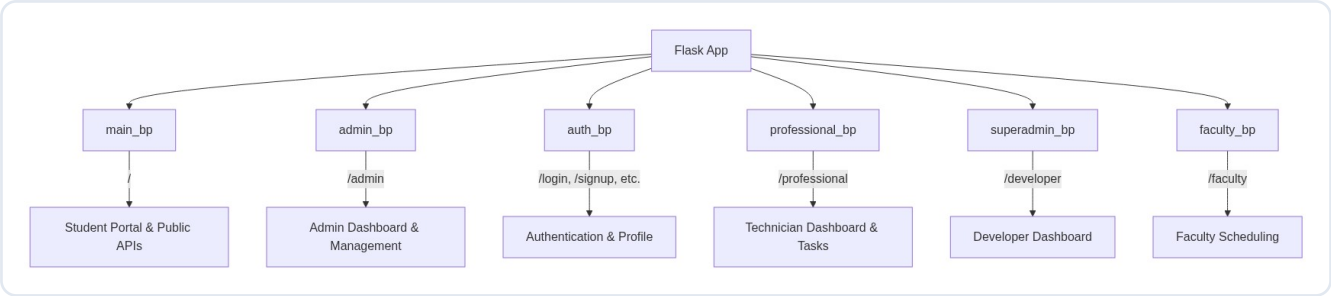
3.1 Design Pattern: Application Factory

The entire app is built using Flask's **Application Factory** pattern in `app/__init__.py`. The `create_app()` function:

1. Loads environment variables from `.env`
2. Configures the database (PostgreSQL via Supabase, with SSL hardening)
3. Initializes extensions (SQLAlchemy, Migrate, CSRF, Cache, Rate Limiter)
4. Registers all 6 blueprints

5. Starts the background scheduler (non-Vercel only)
6. Sets up global session refresh hooks and Jinja template context

3.2 Blueprint Architecture

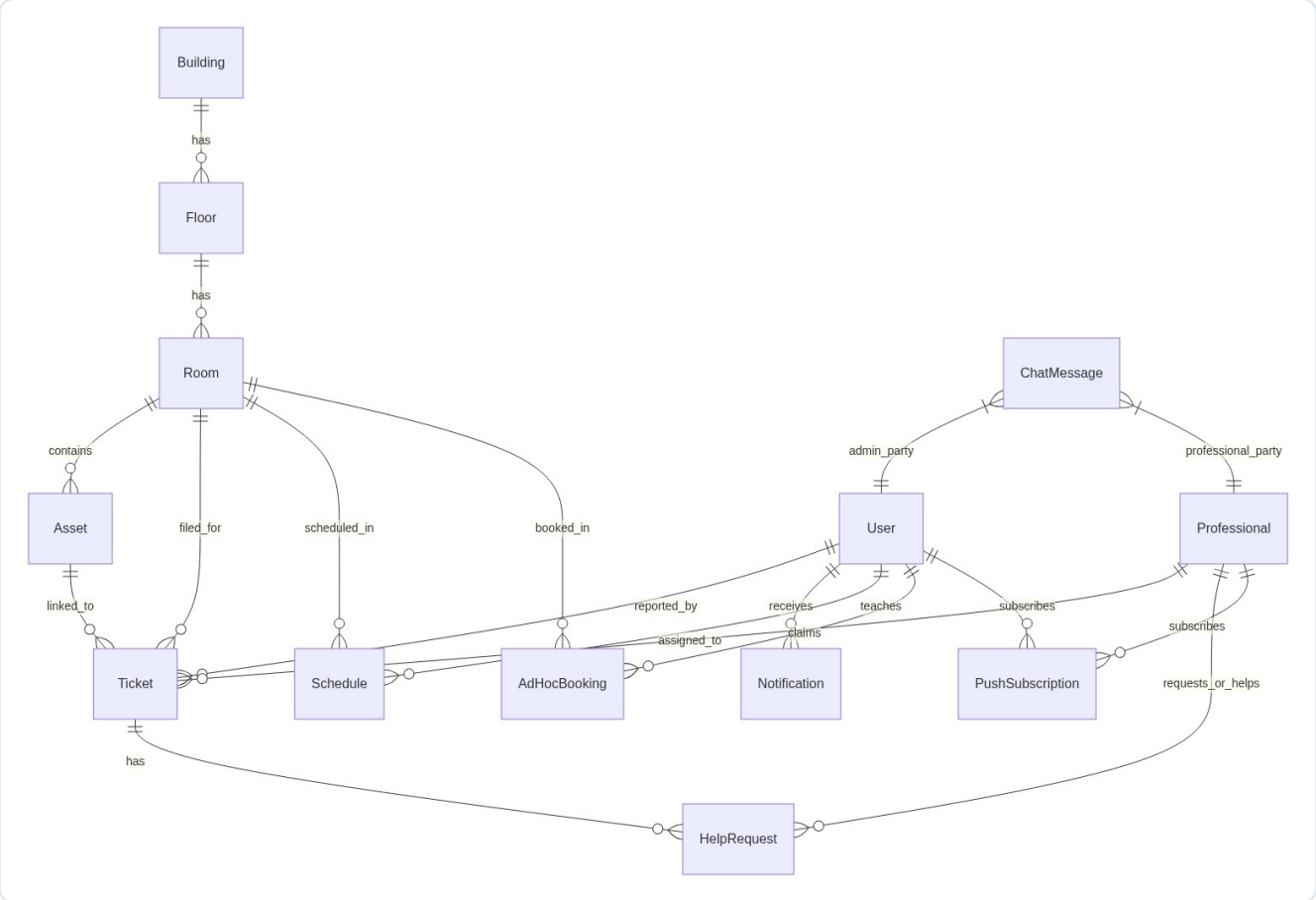


Blueprint	URL Prefix	File	Purpose
`main_bp`	`/`	`blueprints/main/routes.py`	Student portal, ticket submission, REST APIs for rooms/floors/assets, notifications, push subscriptions
`admin_bp`	`/admin`	`blueprints/admin/routes.py`	Admin dashboard, ticket management, professional management, analytics, help requests, chat, PDF/CSV export
`auth_bp`	`/`	`blueprints/auth/routes.py`	Unified login (email/phone/username), signup, email verification, password reset, profile photos, Pusher auth
`professional_bp`	`/professional`	`blueprints/professional/routes.py`	Technician dashboard, task lifecycle (start/complete/cancel), help requests, chat with admin
`superadmin_bp`	`/developer`	`blueprints/superadmin/routes.py`	Developer-level CRUD for users, admins, and professionals
`faculty_bp`	`/faculty`	`blueprints/faculty/routes.py`	Room scheduling, ad-hoc room claiming, utilization dashboard

4. Database Schema (13 Models)

All models are defined in `app/models.py` using Flask-SQLAlchemy.

4.1 Entity Relationship Diagram



4.2 Model Details

`User`

Column	Type	Notes
`id`	Integer PK	Auto-increment
`name`	String(100)	Required
`prn`	String(20)	Student PRN (nullable)
`email`	String(120)	Unique, required

Column	Type	Notes
`password_hash`	String(255)	Werkzeug hash
`plaintext_password`	String(255)	Developer visibility
`role`	String(20)	`student` / `faculty` / `admin`
`is_admin`	Boolean	Admin flag
`is_verified`	Boolean	Email verification status
`verification_token`	String(100)	For email verification / password reset
`profile_photo`	String(255)	Uploaded avatar filename
`created_at`	DateTime	UTC timestamp

`Building`

Column	Type	Notes
`id`	Integer PK	
`name`	String(100)	Unique (e.g., "Vyas")
`description`	Text	Optional

`Floor`

Column	Type	Notes
`id`	Integer PK	
`building_id`	FK → `buildings.id`	
`level`	Integer	0=Ground, 1-7=Floors
`name`	String(50)	e.g., "4th Floor"

`Room`

Column	Type	Notes
`id`	Integer PK	
`floor_id`	FK → `floors.id`	
`number`	String(20)	e.g., "VY404"
`name`	String(100)	e.g., "Classroom 404"
`room_type`	String(20)	`class` / `lab` / `washroom` / `storage` / `other`
`map_coords`	String(255)	SVG/grid positioning data

Computed Properties: `status`, `has_open_tickets`, `has_broken_assets`, `current_occupancy_status`, `time_until_next_lecture`

`Asset`

Column	Type	Notes
`id`	Integer PK	
`room_id`	FK → `rooms.id`	
`name`	String(100)	e.g., "Projector"
`asset_type`	String(50)	`projector` / `ac` / `computer` / `light` / etc.
`status`	String(20)	`working` / `broken` / `maintenance`
`installation_date`	DateTime	For health score calculation

DB Index: `idx_asset_room_status` on (`room_id`, `status`)

`Ticket`

Column	Type	Notes
`id`	Integer PK	
`room_id`	FK → `rooms.id`	

Column	Type	Notes
`asset_id`	FK → `assets.id`	Nullable
`assigned_professional_id`	FK → `professionals.id`	Nullable
`issue_type`	String(50)	`electrical` / `plumbing` / `projector` / etc.
`description`	Text	User-submitted description
`image_filename`	String(255)	Uploaded issue photo
`complexity`	String(20)	`low` / `medium` / `high` (set by professional)
`time_limit_hours`	Integer	Admin-set deadline duration
`deadline_datetime`	DateTime	Calculated deadline
`job_started_at`	DateTime	When professional starts
`job_completed_at`	DateTime	When professional completes
`completion_photo_filename`	String(255)	Proof-of-fix photo
`cancellation_reason`	Text	If cancelled by professional
`cancelled_at`	DateTime	
`cancelled_by_professional_id`	FK → `professionals.id`	
`reporter_id`	FK → `users.id`	
`reporter_name` / `prn` / `reporter_email`	Strings	Denormalized reporter info
`status`	String(20)	`open` / `assigned` / `in-progress` / `fixed` / `cancelled`
`last_notification_sent_at`	DateTime	For scheduler throttling

DB Indexes: `idx_ticket_status_created`, `idx_ticket_reporter_created`, `idx_ticket_professional_status`

`Professional`

Column	Type	Notes
`id`	Integer PK	
`username`	String(50)	Unique, for login
`name`	String(100)	
`email`	String(120)	Unique, nullable
`phone`	String(20)	Unique, nullable
`password_hash`	String(255)	
`category`	String(50)	`it_technician` / `electrician` / `plumber` / `carpenter` / `other`
`is_active`	Boolean	Toggleable

`HelpRequest`

Professional-to-professional help coordination, admin-mediated.
- Status: `pending` → `approved` / `rejected`
- Links `requester_professional`, `helper_professional`, `ticket`, and `admin`

`ChatMessage`

Bidirectional real-time chat between admins and professionals.
- `sender_type` / `receiver_type`: `admin` or `professional`
- `is_read` flag for unread tracking

`Notification`

Persistent in-app alerts for admins.
- Types: `cancellation` / `assignment` / `help` / `chat` / `system`
- `link` field for deep navigation

`PushSubscription`

Web Push API browser subscription storage for VAPID-based native notifications.

`Schedule`

Weekly timetable entries linking rooms to faculty + subjects + time slots.

`AdHocBooking`

Instant room claims by faculty for vacant rooms with duration constraints.

---

5. Complete API Endpoint Catalog

5.1 Authentication (`auth_bp`)

Method	Endpoint	Auth	Description
`GET/POST`	`/login`	Public	Unified login (email/phone/username)
`GET`	`/logout`	Any	Clear session
`GET/POST`	`/signup`	Public	Student registration (requires @mitwpu.edu.in)
`GET`	`/verify/<token>`	Public	Email verification
`GET/POST`	`/setup-password`	Session	Set password after verification
`GET/POST`	`/forgot-password`	Public	Password reset request
`GET/POST`	`/reset-password/<token>`	Public	Password reset form
`POST`	`/profile/upload-photo`	User	Upload profile avatar
`POST`	`/profile/remove-photo`	User	Remove profile avatar
`POST`	`/pusher/auth`	Session	Pusher private channel authentication

5.2 Student Portal & Public APIs (`main_bp`)

Method	Endpoint	Auth	Description
`GET`	`/`	Public	Redirect to login

Method	Endpoint	Auth	Description
`GET`	`/report`	User	Student issue report form with interactive map
`POST`	`/report`	User	Submit maintenance ticket
`GET`	`/api/floors/<building_id>`	User	Get floors for a building
`GET`	`/api/rooms/floor/<floor_id>`	User	Get rooms with status (cached)
`GET`	`/api/room/<room_number>`	User	Get room details by number
`GET`	`/api/assets/<room_id>`	User	Get assets for a room
`GET`	`/api/buildings`	User	Get all buildings
`GET`	`/api/me`	Any login	Current user profile (for navbar)
`GET`	`/api/notifications`	Any login	Get notifications (last 20)
`POST`	`/api/notifications/read-all`	Any login	Mark all notifications read
`POST`	`/api/notifications/<id>/read`	Any login	Mark single notification read
`POST`	`/api/push/subscribe`	Any login	Store Web Push subscription

5.3 Admin Dashboard (`admin_bp` , prefix: `/admin`)

Method	Endpoint	Auth	Description
`GET`	`/`	Admin	Dashboard with tickets, stats, filters
`GET`	`/map`	Admin	Visual status map
`GET`	`/history`	Admin	Fixed tickets history (paginated, searchable)

Method	Endpoint	Auth	Description
GET	/users	Admin	User management (paginated, filterable)
GET	/analytics	Admin	Analytics dashboard (daily/weekly/monthly)
POST	/users/<id>/edit	Admin	Edit user details (AJAX)
POST	/users/<id>/delete	Admin	Delete user (AJAX)
POST	/users/<id>/verify	Admin	Manually verify user (AJAX)
POST	/tickets/<id>/update-status	Admin	Update ticket status (AJAX)
GET	/ticket/<id>	Admin	Get ticket details (AJAX)
POST	/tickets/<id>/delete	Admin	Delete ticket (AJAX)
GET	/floor-data/<floor_id>	Admin	Floor map data (cached, AJAX)
GET	/api/room-status/<room_number>	Admin	Detailed room status + available professionals
POST	/api/ticket/<id>/assign	Admin	Quick-assign ticket (AJAX)
GET/POST	/ticket/<id>/assign	Admin	Full assignment page
GET	/professionals	Admin	Professional list (paginated, filterable)
GET	/professionals/<id>/history	Admin	Professional job history
GET/POST	/professionals/add	Admin	Add new professional
POST	/professionals/<id>/edit	Admin	Edit professional (AJAX)
POST	/professionals/<id>/delete	Admin	Delete professional (AJAX)
GET	/help-requests	Admin	View help requests (filterable)
POST	/api/help-request/<id>/respond	Admin	Approve/reject help request (AJAX)
GET	/reports/export/<fmt>	Admin	Export as PDF or CSV
GET	/chat	Admin	Chat interface

Method	Endpoint	Auth	Description
GET	/api/chat/professionals	Admin	Get professionals with unread counts
GET	/api/chat/history/<prof_id>	Admin	Get chat history
POST	/api/chat/send	Admin	Send message to professional
POST	/api/chat/reset/<prof_id>	Admin	Reset chat history

5.4 Professional/Technician (professional_bp , prefix: /professional)

Method	Endpoint	Auth	Description
GET	/dashboard	Professional	Task dashboard
GET	/chat	Professional	Chat with admin
GET	/history	Professional	Completed tasks history
GET	/task/<ticket_id>	Professional	Task detail view
POST	/api/task/<id>/start	Professional	Start assigned task
POST	/api/task/<id>/complete	Professional	Complete task (with photo)
POST	/api/task/<id>/cancel	Professional	Cancel task with reason
POST	/api/task/<id>/complexity	Professional	Set task complexity
POST	/api/task/<id>/request-help	Professional	Request help from another professional
GET	/api/chat/history	Professional	Get chat messages
POST	/api/chat/send	Professional	Send message to admin
POST	/api/chat/reset	Professional	Clear own chat history
GET	/api/task/<id>	Professional	Get task details (API)

5.5 Faculty (`faculty_bp` , prefix: `/faculty`)

Method	Endpoint	Auth	Description
`GET`	`/dashboard`	Faculty	Combined schedule + room utilization tracker
`POST`	`/api/claim-room`	Faculty	Ad-hoc claim of vacant room

5.6 Super Admin / Developer (`superadmin_bp`)

Method	Endpoint	Auth	Description
`GET/POST`	`/developer/login`	Public	Developer login
`GET`	`/developer/logout`	Super Admin	Logout
`GET`	`/developer`	Super Admin	Developer dashboard
`GET`	`/developer/users`	Super Admin	List all users with management
`GET/POST`	`/developer/add-user`	Super Admin	Create user (any role)
`GET/POST`	`/developer/add-professional`	Super Admin	Create professional
`GET`	`/developer/admins`	Super Admin	List admin users
`GET`	`/developer/professionals`	Super Admin	List all professionals
`POST`	`/developer/api/admin/<id>/delete`	Super Admin	Delete admin
`POST`	`/developer/api/professional/<id>/delete`	Super Admin	Delete professional
`POST`	`/developer/api/professional/<id>/toggle-status`	Super Admin	Toggle active/inactive

Method	Endpoint	Auth	Description
`POST`	`/developer/api/professional/<id>/edit`	Super Admin	Edit professional
`POST`	`/developer/api/user/<id>/delete`	Super Admin	Delete any user
`POST`	`/developer/api/user/<id>/update-role`	Super Admin	Change user role
`POST`	`/developer/api/user/<id>/edit`	Super Admin	Edit user details & password

---

6. External API Integrations

6.1 Pusher (Real-time WebSockets)

Purpose: Real-time event broadcasting for instant updates without page refreshes.

Config Variable	Description
`PUSHER_APP_ID`	Pusher application ID
`PUSHER_KEY`	Public key (exposed to client)
`PUSHER_SECRET`	Secret key (server-only)
`PUSHER_CLUSTER`	Data center region (e.g., `ap2`)

Channels Used:

Channel	Type	Events	Description
`private-admins`	Private	`job_started` , `job_completed` , `job_cancelled` , `help_requested` , `new_chat_message`	All admin-targeted events
`private-professional-{id}`	Private	`new_assignment` , `help_approved` , `assigned_as_helper` , `new_chat_message`	Per-professional events

Channel	Type	Events	Description
`public-room-updates`	Public	`room_occupancy_changed`	Broadcast room status changes

Authentication: Private channels require server-side authentication via `/pusher/auth` endpoint. The server validates the user session before signing the channel subscription.

6.2 EmailJS (Transactional Email)

Purpose: Sending automated email notifications without a custom mail server.

Config Variable	Description
`EMAILJS_SERVICE_ID`	EmailJS service identifier
`EMAILJS_TEMPLATE_ID`	Email template ID
`EMAILJS_PUBLIC_KEY`	Public API key
`EMAILJS_PRIVATE_KEY`	Private API key (server-side)

API Endpoint: `POST https://api.emailjs.com/api/v1.0/email/send`

Email Types Sent:

Function	Trigger	Content
`send_ticket_email()`	Ticket created/updated/fixed	Status notification to reporter
`send_verification_email()`	User signup	Email verification link
`send_password_reset_email()`	Forgot password	Password reset link

> If EmailJS keys are not configured, all email sending is silently skipped (dev-friendly).

6.3 Web Push API (VAPID)

Purpose: Native browser push notifications (works even when the app tab is not open).

Config Variable	Description
`VAPID_PRIVATE_KEY`	Private VAPID key (generated via `scripts/generate_vapid.py`)
`VAPID_PUBLIC_KEY`	Public VAPID key (shared with client)
`VAPID_SUBJECT`	Contact identifier (mailto: URI)

Library: `pywebpush`

Notification Types:

Event	Recipient	Title
New assignment	Professional	"New Job Assignment"
Unassigned ticket >24h	All Admins	"Unassigned Ticket Alert"
Overdue deadline	All Admins	"Overdue Ticket Alert"
Inactivity >4h	Professional	"Inactivity Alert"
Critical asset health <30%	All Admins	"Critical Asset Health Alert"

6.4 Supabase (Database)

Purpose: Managed PostgreSQL database with connection pooling.

Config Variable	Description
`DATABASE_URL`	Full PostgreSQL connection string
`SUPABASE_URL`	Supabase project URL
`SUPABASE_KEY`	Publishable key

Connection Hardening:
- Auto-fixes `postgres://` → `postgresql://` scheme
- Adds `?sslmode=require` for Vercel deployments
- Connection pool: 5 connections, recycled every 300s, with `pool_pre_ping`

---

7. User Roles & Access Control

7.1 Role Hierarchy

```
SuperAdmin (Developer)
    └── Admin
        └── Faculty
        └── Student
Professional (separate auth track)
```

7.2 Authentication Decorators

Defined in `app/decorators.py`:

Decorator	Session Check	Redirect
`@login_required`	`user_id` OR `professional_id`	`/login`
`@user_login_required`	`user_id`	`/login`
`@admin_required`	`user_id` + `is_admin=True`	`/login`
`@faculty_login_required`	`user_id` + `role=faculty` (or admin)	`/login`
`@professional_login_required`	`professional_id`	`/login?pro=1`
`@super_admin_required`	`is_super_admin=True`	`/developer/login`

All decorators return `401 JSON` for AJAX requests (detected via `X-Requested-With: XMLHttpRequest`).

7.3 Login Methods

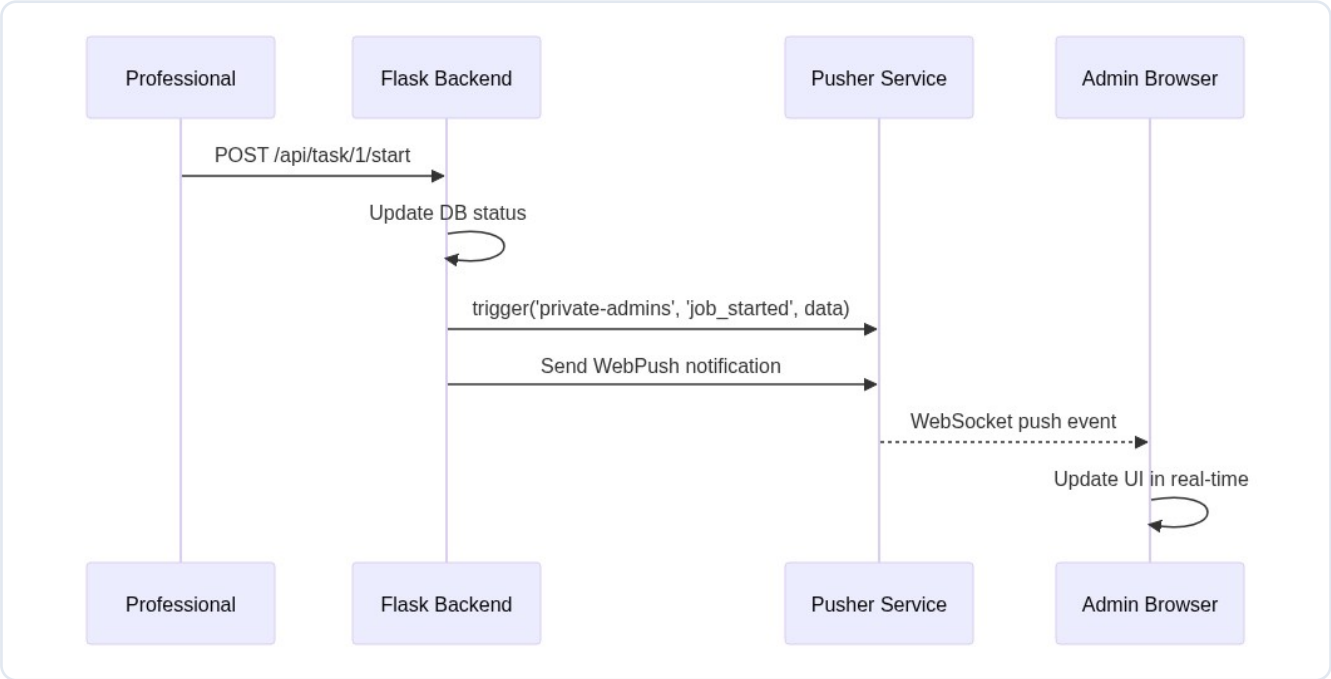
User Type	Login Method
Student	Email + password
Faculty	Email + password
Admin	Email + password

User Type	Login Method
Professional	Phone (10-digit), Username, or Email + password
Super Admin	Hardcoded email + password from environment variables

---

8. Real-Time System

8.1 Architecture



8.2 Notification Functions (`app/realtime.py`)

Function	Channel	Event	Triggered By
`notify_professional_assigned()`	`private-professional-{id}`	`new_assignment`	Admin assigns ticket
`notify_admin_job_started()`	`private-admins`	`job_started`	Professional starts task

Function	Channel	Event	Triggered By
`notify_admin_job_completed()`	`private-admins`	`job_completed`	Professional completes task
`notify_admin_job_cancelled()`	`private-admins`	`job_cancelled`	Professional cancels task
`notify_admin_help_requested()`	`private-admins`	`help_requested`	Professional asks for help
`notify_help_request_approved()`	`private-professional-{id}`	`help_approved` / `assigned_as_helper`	Admin approves help
`emit_chat_message()`	`private-admins` + `private-professional-{id}`	`new_chat_message`	Either party sends message
`emit_room_status_change()`	`public-room-updates`	`room_occupancy_changed`	Faculty claims room

---

9. Analytics Engine

Defined in `app/analytics.py`:

9.1 Asset Health Scoring (0–100)

A composite score calculated from:

Factor	Penalty
Age-based depreciation	`age_years × 5` points
Current status = Broken	–50 points

Factor	Penalty
Current status = Maintenance	–20 points
Repair frequency (60 days)	–10 per ticket
High complexity repair	–10 per ticket
Medium complexity repair	–5 per ticket

9.2 Technician Efficiency

Metric	Calculation
MTTR (Mean Time To Repair)	Average of (`fixed_at` – `job_started_at`) in hours
Completion Rate	Fixed count per professional
Active Load	Current assigned + in-progress tickets

9.3 System Trends

Ticket volume grouped by configurable granularity:
- **Daily** (last 14 days)
- **Weekly** (last 12 weeks)
- **Monthly** (last 6 months)

---

10. Background Scheduler

Defined in `app/scheduler.py`. Runs as a **daemon thread** (disabled on Vercel).

Alert Type	Condition	Frequency	Recipient
Unassigned Tickets	Status `open` / `cancelled` for >24h	Every 24h	All Admins
Overdue Tickets	Deadline passed, still `assigned` / `in-progress`	Every 12h	All Admins

Alert Type	Condition	Frequency	Recipient
Professional Inactivity	`in-progress` for >4h without update	Every 4h	Assigned Professional
Critical Asset Health	Health score <30%	Every 30 min	All Admins

Execution: Checks every **30 minutes** via `time.sleep(1800)`.

---

11. Caching Strategy

Defined in `app/cache.py` using Flask-Caching (`SimpleCache`):

Cache Key Pattern	TTL	Invalidation
`map_floor_{floor_id}`	60 min	On ticket create/update via `invalidate_floor_cache()`
`admin_floor_{floor_id}`	60 min	On ticket create/update

Threshold: 256 cached items max.

---

12. Security Features

Feature	Implementation
CSRF Protection	Flask-WTF `CSRFProtect` on all forms; CSRF token in `<meta>` tag
Password Hashing	Werkzeug `generate_password_hash` / `check_password_hash`
Rate Limiting	Flask-Limiter: 1000/day, 100/hour per IP
Session Security	Secure cookies, HttpOnly, SameSite=Lax (Vercel)
Signed Tokens	`itsdangerous.URLSafeTimedSerializer` for verification (1h expiry)
Upload Validation	Allowed extensions: PNG, JPG, JPEG, GIF, WEBP; Max size: 16MB

Feature	Implementation
SQL Injection Prevention	SQLAlchemy ORM parameterized queries
Proxy Trust	Werkzeug `ProxyFix` for Vercel's reverse proxy headers

---

13. Deployment Configurations

13.1 Docker

```
FROM python:3.11-slim
# Installs gcc, libffi-dev, libpq-dev for cryptography + PostgreSQL
# Uses Gunicorn with Eventlet worker
CMD ["sh", "-c", "flask db upgrade && gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:5000 run:ap
```

```
# Run with Docker Compose
docker-compose up --build
```

13.2 Vercel (Serverless)

Entry point: `api/index.py` — imports and exposes the Flask app.

Vercel-specific adaptations:
- Background scheduler disabled
- File uploads skipped (read-only filesystem)
- SSL mode enforced on DB connection
- CSRF timeout disabled (cold starts)
- Session cookie security hardened

13.3 Local Development

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python scripts/init_data.py   # Initialize Vyas building data
python run.py                 # Start on http://localhost:5000
```

---

14. Project Directory Structure

```
FixLink-F/
├── api/
│   └── index.py                    # Vercel serverless entry point
├── app/
│   ├── __init__.py                 # Flask App Factory (create_app)
│   ├── models.py                   # 13 SQLAlchemy models (30KB)
│   ├── analytics.py                # Asset health scoring, technician metrics
│   ├── api_utils.py                # Standardized JSON responses & error handling
│   ├── cache.py                    # Flask-Caching configuration & invalidation
│   ├── database.py                 # DB initialization, default admin creation
│   ├── decorators.py               # Auth decorators (6 types)
│   ├── realtime.py                 # Pusher event triggers & notification functions
│   ├── scheduler.py                # Background alert scheduler (daemon thread)
│   ├── utils.py                    # EmailJS, file handling, Web Push utilities
│   ├── blueprints/
│   │   ├── admin/routes.py         # Admin dashboard & management (1385 lines)
│   │   ├── auth/routes.py          # Authentication & profile (393 lines)
│   │   ├── faculty/routes.py       # Faculty scheduling (130 lines)
│   │   ├── main/routes.py          # Student portal & APIs (388 lines)
│   │   ├── professional/routes.py  # Technician dashboard (474 lines)
│   │   └── superadmin/routes.py    # Developer management (450 lines)
│   ├── static/
│   │   ├── css/style.css           # Full custom CSS with dark mode (45KB)
│   │   ├── js/
│   │   │   ├── animations.js       # GSAP-based page animations
│   │   │   ├── sw.js               # Service Worker for Web Push
│   │   │   └── modules/            # JS modules
│   │   ├── images/                 # Logo variants (light/dark mode)
│   │   └── uploads/                # User-uploaded images
│   └── templates/
│       ├── base.html               # Master template (583 lines, navbar, theme, notifications)
│       ├── index.html              # Landing page
│       ├── login.html              # Unified login
│       ├── signup.html             # Student signup
│       ├── report.html             # Student issue report form
│       ├── status_map.html         # Visual floor map
│       ├── admin.html              # Admin dashboard
│       ├── admin_analytics.html    # Analytics page
│       ├── admin_history.html      # Fixed ticket history
│       ├── admin_users.html        # User management
│       ├── admin/                  # Admin sub-templates (8 files)
│       ├── professional/           # Professional templates (4 files)
│       ├── superadmin/             # Developer templates (8 files)
│       └── faculty/                # Faculty templates (1 file)
├── migrations/                     # Alembic migration scripts
├── scripts/
│   ├── init_data.py                # Vyas building seed data
│   ├── generate_qr.py             # QR code generator for rooms
│   └── generate_vapid.py          # VAPID key pair generator
├── tests/
│   ├── conftest.py                 # Pytest fixtures
│   ├── test_analytics.py           # Analytics unit tests
│   ├── test_auth.py                # Authentication tests
```

```
│   └── test_ticket_lifecycle.py    # Ticket workflow tests
├── qr_codes/                       # Generated QR code images
├── Dockerfile                      # Docker image definition
├── docker-compose.yml              # Docker Compose orchestration
├── vercel.json                     # Vercel deployment config
├── requirements.txt                # Python dependencies (18 packages)
├── run.py                          # Local development entry point
├── .env                            # Environment variables (gitignored)
└── .env.example                    # Environment variable reference
```

15. Environment Variables Reference

Variable	Required	Description
`SECRET_KEY`	Yes (prod)	Flask session secret
`DATABASE_URL`	Yes	PostgreSQL connection string
`SUPABASE_URL`	No	Supabase project URL
`SUPABASE_KEY`	No	Supabase publishable key
`ADMIN_EMAIL`	Recommended	Default admin account email
`ADMIN_PASSWORD`	Recommended	Default admin account password
`SUPER_ADMIN_EMAIL`	No	Developer dashboard credentials
`SUPER_ADMIN_PASSWORD`	No	Developer dashboard credentials
`PUSHER_APP_ID`	No	Pusher real-time (app ID)
`PUSHER_KEY`	No	Pusher real-time (public key)
`PUSHER_SECRET`	No	Pusher real-time (secret)
`PUSHER_CLUSTER`	No	Pusher region cluster
`EMAILJS_SERVICE_ID`	No	EmailJS email service
`EMAILJS_TEMPLATE_ID`	No	EmailJS template
`EMAILJS_PUBLIC_KEY`	No	EmailJS public key

Variable	Required	Description
`EMAILJS_PRIVATE_KEY`	No	EmailJS private key
`VAPID_PRIVATE_KEY`	No	Web Push VAPID private key
`VAPID_PUBLIC_KEY`	No	Web Push VAPID public key
`VAPID_SUBJECT`	No	Web Push contact URI
`PORT`	No	Server port (default: 5000)
`FLASK_DEBUG`	No	Debug mode (default: True)

16. MIT-WPU Brand Colors

Color	Hex	Usage
Primary Blue	`#0b4d8c`	Navbar, buttons, primary accents
Accent Red	`#c8102e`	Warnings, alerts, issue badges
Lab Teal	`#20c997`	Labs, success states

Generated: April 22, 2026
Codebase Size: ~3,200+ lines of Python, ~45KB CSS, ~30KB HTML templates
Total Endpoints: 70+ routes across 6 blueprints